



[Figure 2-4:](#) Rule 3 of Kurushima's algorithm

This rule is followed when we are crossing the antidiagonal. If we cross from the bottom right to the top left, we can follow the inverse of this rule, in which x is transformed to $x + n - 1, \text{ mod } n^2$.

We can write a simple implementation of Rule 1 in Python by defining a function that takes x and n as its arguments and returns $(x+n)\%n^2$:

```
def rule1(x,n):
    return((x + n)%n**2)
```

We can try this out with the central entry in the Luo Shu square. Remember, the Luo Shu square is a 3×3 square matrix, so $n = 3$. The central entry of the Luo Shu square is 5. The entry below and to the left of this entry is 8, and if we have implemented our `rule1()` function correctly we'll get an 8 when we run the following line:

```
print(rule1(5,3))
```

You should see an 8 in the Python console. Our `rule1()` function seems to work as intended. However, we could improve it by enabling it to go “in reverse,” determining not only the entry on the bottom left of a given entry, but also the entry to the top right (that is, being able to go from 8 to 5 in addition to going from 5 to 8). We can make this improvement by adding one more argument to the function. We'll call our new argument `upright`, and it will be a `True/False` indicator of whether we're looking for the entry up and to the right of x . If not, we will by default look for the entry to the bottom left of x :

```
def rule1(x,n,upright):
    return((x + ((-1)**upright) * n)%n**2)
```

In a mathematical expression, Python will interpret `True` as 1 and `False` as 0. If `upright` is `False`, our function will return the same value as before, since